B1B13PPS

Průmyslové počítačové systémy

Michal Brejcha
brejcmic@fel.cvut.cz

ČVUT v Praze, FEL

Praha, 2025

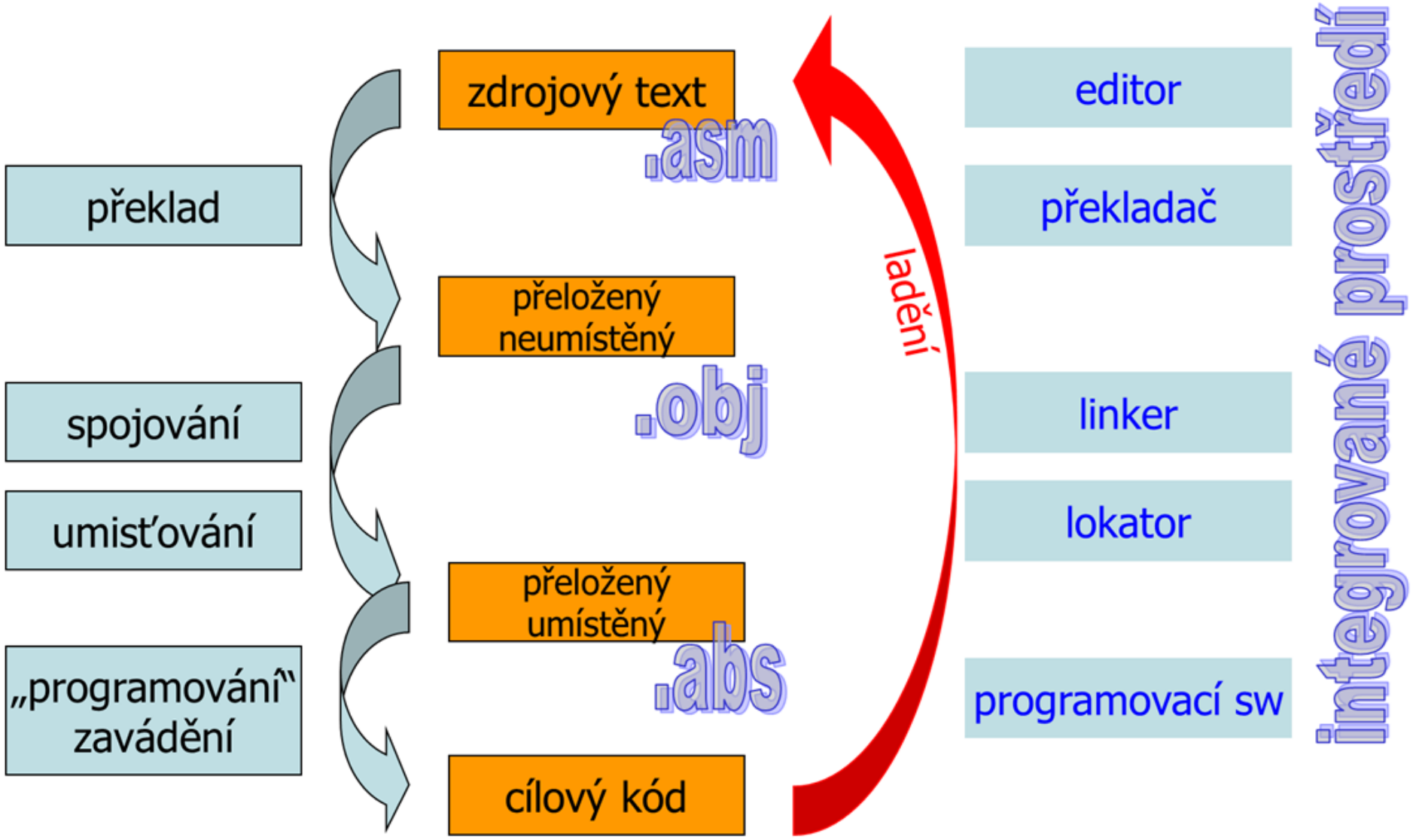
Obsah

1. Programování - Build
2. HMI, SCADA a OPC
3. Control Web

Téma

1. Programování - Build
2. HMI, SCADA a OPC
3. Control Web

Build Pipeline



Zdrojový soubor

- je kód, který píše programátor, tj. textový soubor,
- obsahuje popis proměnných, funkcí, skoků, logiky atd.,
- liší se podle programovacího jazyku = způsob popisu,

- zdrojový soubor téhož programu psaný v assembleru se bude významně lišit mezi jednotlivými řadami procesorů - obvykle se liší jak v registrech tak v množině instrukcí,
- pro vyšší programovací jazyky (C, C++, ...) je podobnost zdrojového kódu vyšší - liší se hlavně v registrech a funkcích daného procesoru.

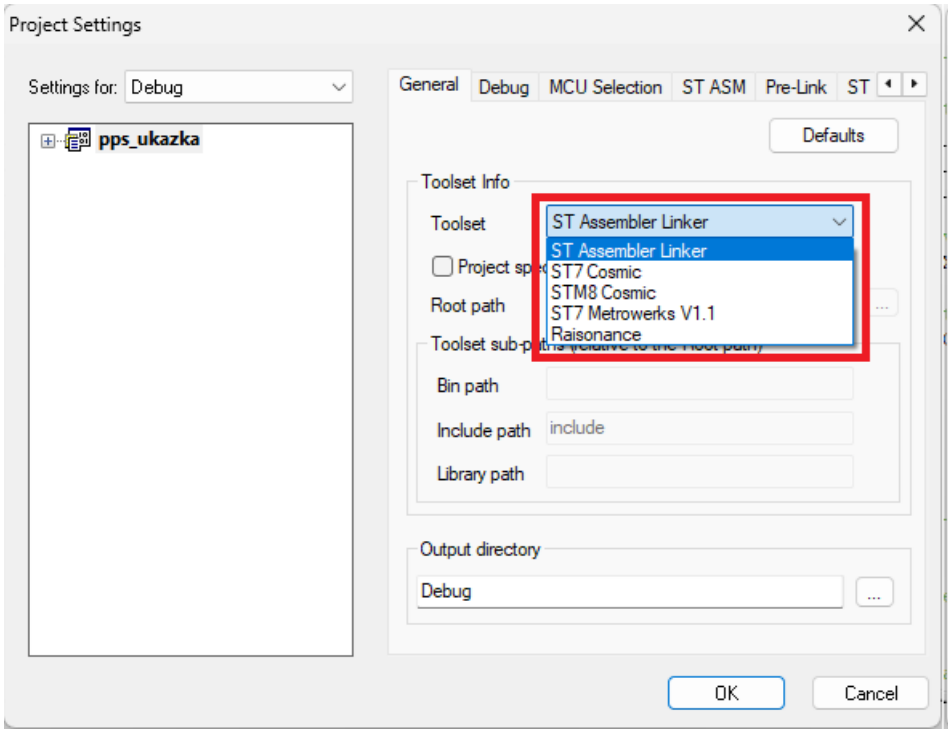
Překladač

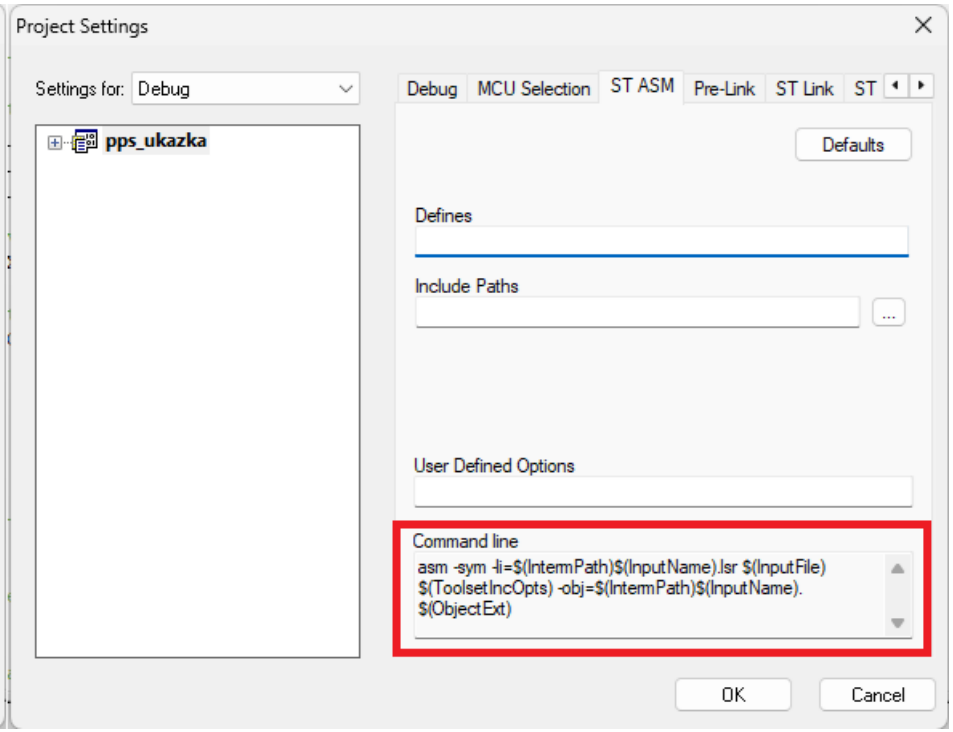
- Převádí zdrojový kód do strojově blízké formy - instrukce procesoru bez adres proměnných (tj. binární soubor),
- kontroluje chyby syntaxe jazyka, kontroluje typy proměnných apod.,
- generuje symboly:
  - názvy funkcí,
  - názvy proměnných,
- výstupem je objektový soubor **.obj** (**.o**) pro každý zdrojový soubor.

Příklad:

$$\text{main.c} \longrightarrow \text{main.o}$$
$$\text{fibonacci.asm} \longrightarrow \text{fibonacci.obj}$$

Překladač STVD





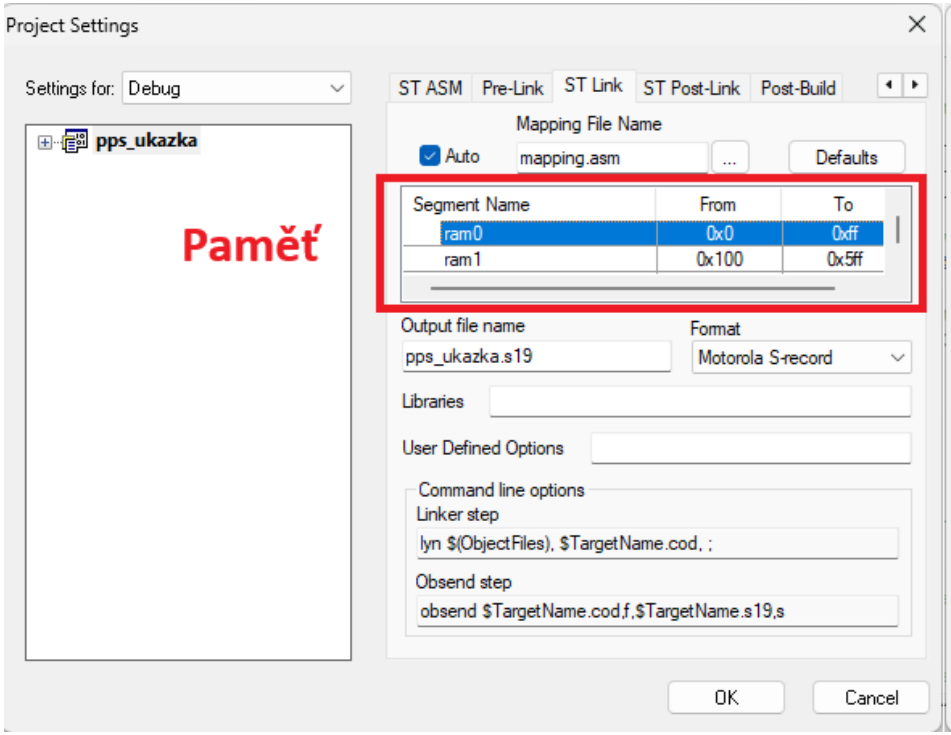
Linker

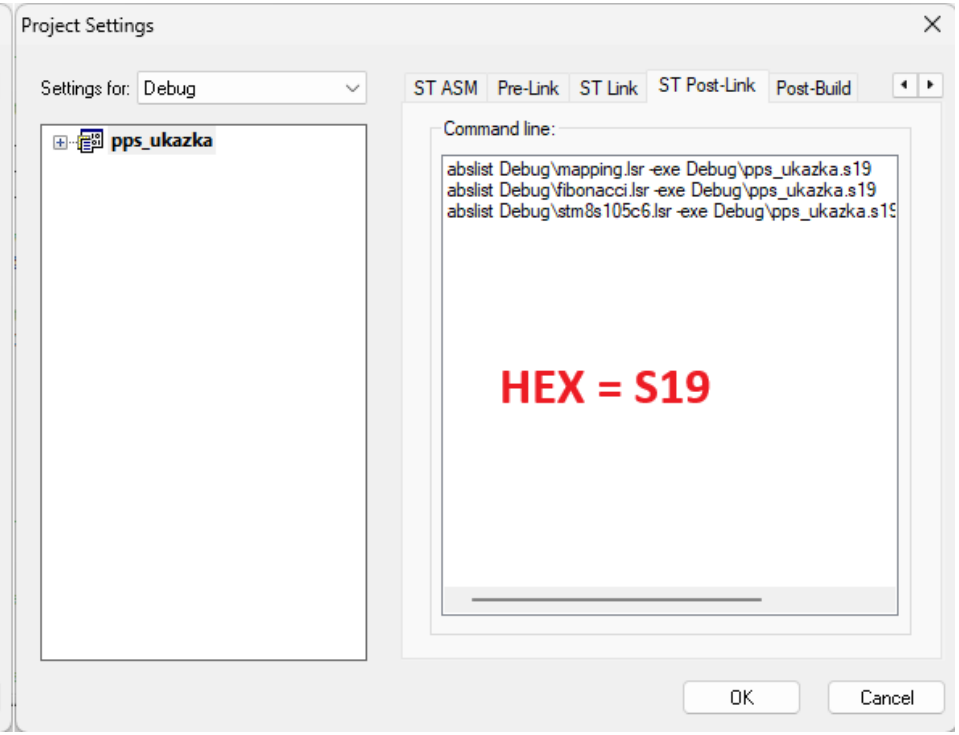
- Pracuje s celým projektem,
- spojuje jednotlivé objektové soubory a vyhrazuje jim adresy v paměti:
  - adresy proměnných,
  - adresy návěstí, ...
- řeší také odkazy mezi soubory a do knihoven,
- výstupem je binární soubor **.hex** (nebo spustitelný soubor **.exe**) pro program.

Příklad:

$$\text{main.o, spi.o} \longrightarrow \text{main.exe}$$

Linker STVD


Project Settings
Settings for: Debug
pps_ukazka
Paměť
ST ASM
Pre-Link
ST Link
ST Post-Link
Post-Build
Mapping File Name
Auto
mapping.asm
Defaults
Segment Name
From
To
ram0
0x0
0xff
ram1
0x100
0x5ff
Output file name
pps_ukazka.s19
Format
Motorola S-record
Libraries
User Defined Options
Command line options
Linker step
lyn $(ObjectFiles), $TargetName.cod, ;
Obsend step
obsend $TargetName.cod,f,$TargetName.s19,s
OK
Cancel


Project Settings
Settings for: Debug
pps_ukazka
ST ASM
Pre-Link
ST Link
ST Post-Link
Post-Build
Command line:
abslist Debug\mapping.lsr -exe Debug\pps_ukazka.s19
abslist Debug\fibonacci.lsr -exe Debug\pps_ukazka.s19
abslist Debug\stm8s105c6.lsr -exe Debug\pps_ukazka.s19
HEX = S19
OK
Cancel

Výstupní hexfile typu MOTOROLA

Program fibonacci.asm

```
1  S00600004844521B
2  S113808035FF500535FF500835FF50073500000116
3  S1138090C60001CD80A843C75005725C0001C6002C
4  S11380A001A10D23EECC808CA1012201814A88CD4F
5  S11380B080A8887B024ACD80A81B016B028484813E
6  S11380C08A3505000090AEC350905A26FC725A00BF
7  S10980D00026F286818007
8  S113800082008080820080D5820080D5820080D565
9  S113801082008 0D5820080D5820080D5820080D500
10 S113802082008 0D5820080D5820080D5820080D5F0
11 S113803082008 0D5820080D5820080D5820080D5E0
12 S113804082008 0D5820080D5820080D5820080D5D0
13 S113805082008 0D5820080D5820080D5820080D5C0
14 S113806082008 0D5820080D5820080D5820080D5B0
15 S113807082008 0D5820080D5820080D5820080D5A0
16 S9030000FC
17
```

Motorola záznam: typ | počet byte

8080 mov PB_ODR, #$FF

Některé typy záznamů:

- S0 - hlavička
- S1 - data, adresa a strojový kód
- S9 - konec souboru

- Zde je to textový soubor, nicméně může být také binární stejně jako objektové soubory.

Téma

1 Programování - Build

2 HMI, SCADA a OPC

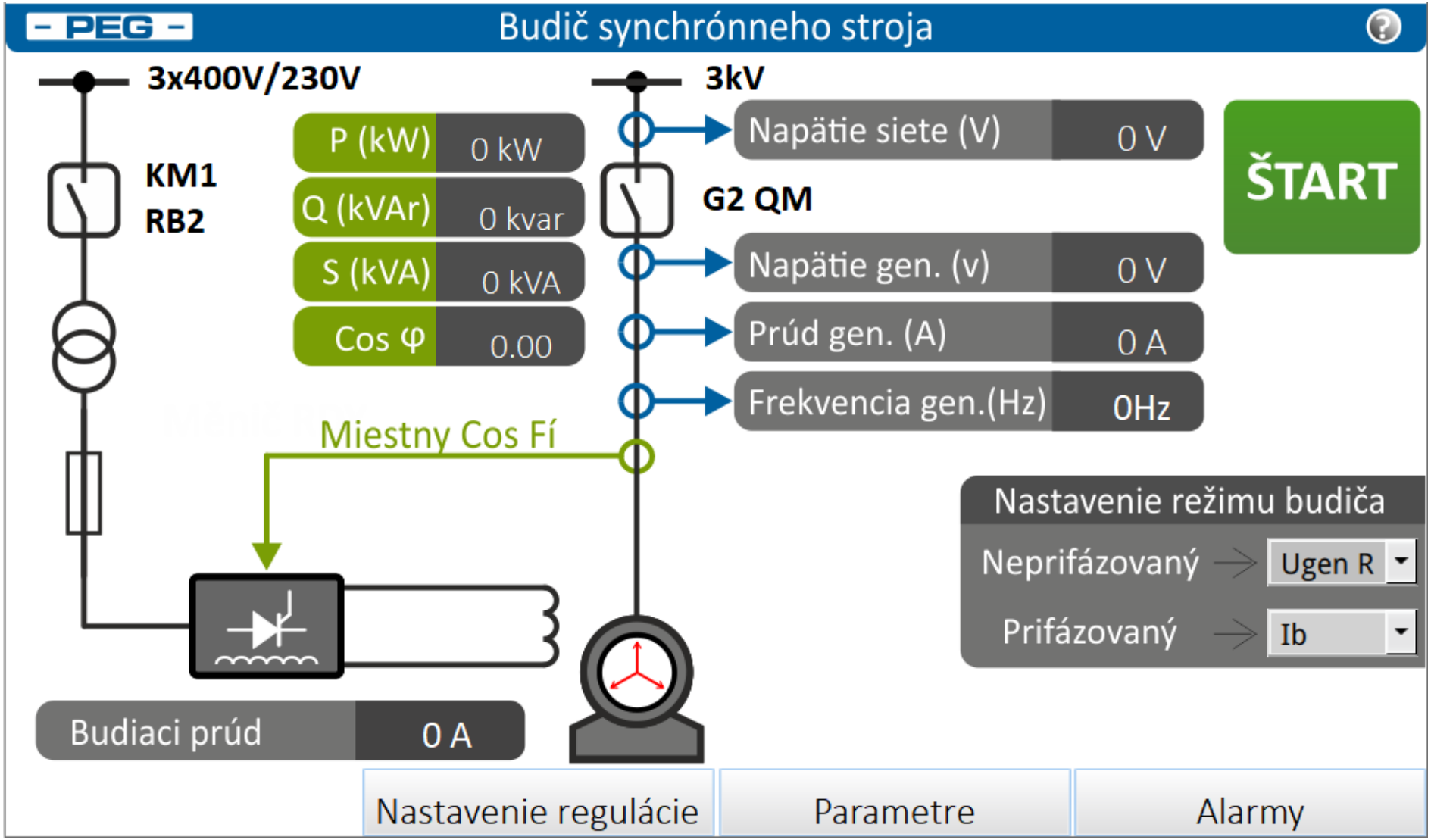
3 Control Web

HMI - lokální dohled

Human **M**achine **I**nterface

- je ovládací rozhraní pro operátora určitého stroje:
  - obrazovka,
  - panel,
  - aplikace,
- často běží přímo na panelu u stroje.

HMI - příklad ovládání budiče

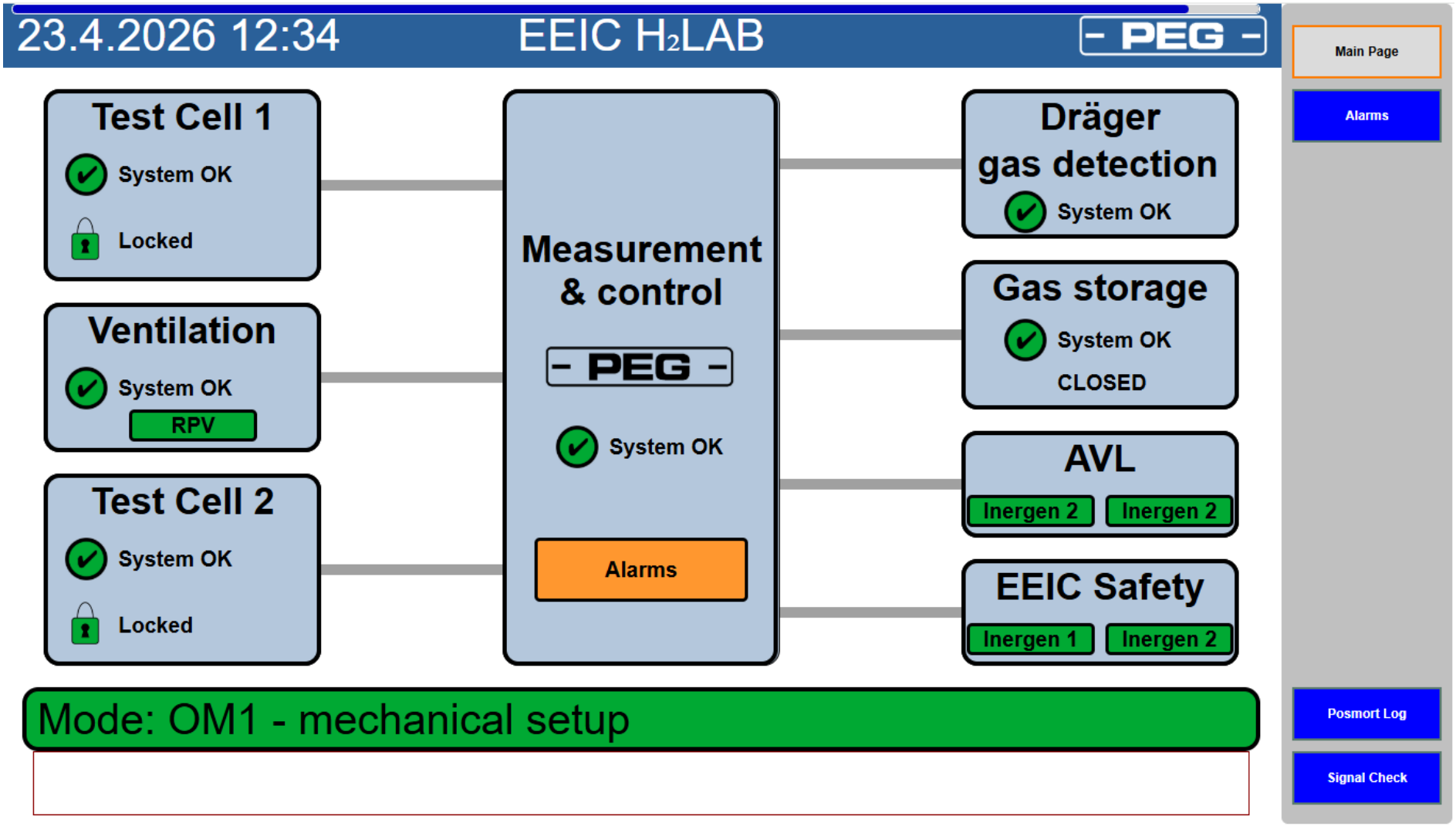


SCADA - celkový dohled

Supervisory **C**ontrol and **D**ata **A**cquisition

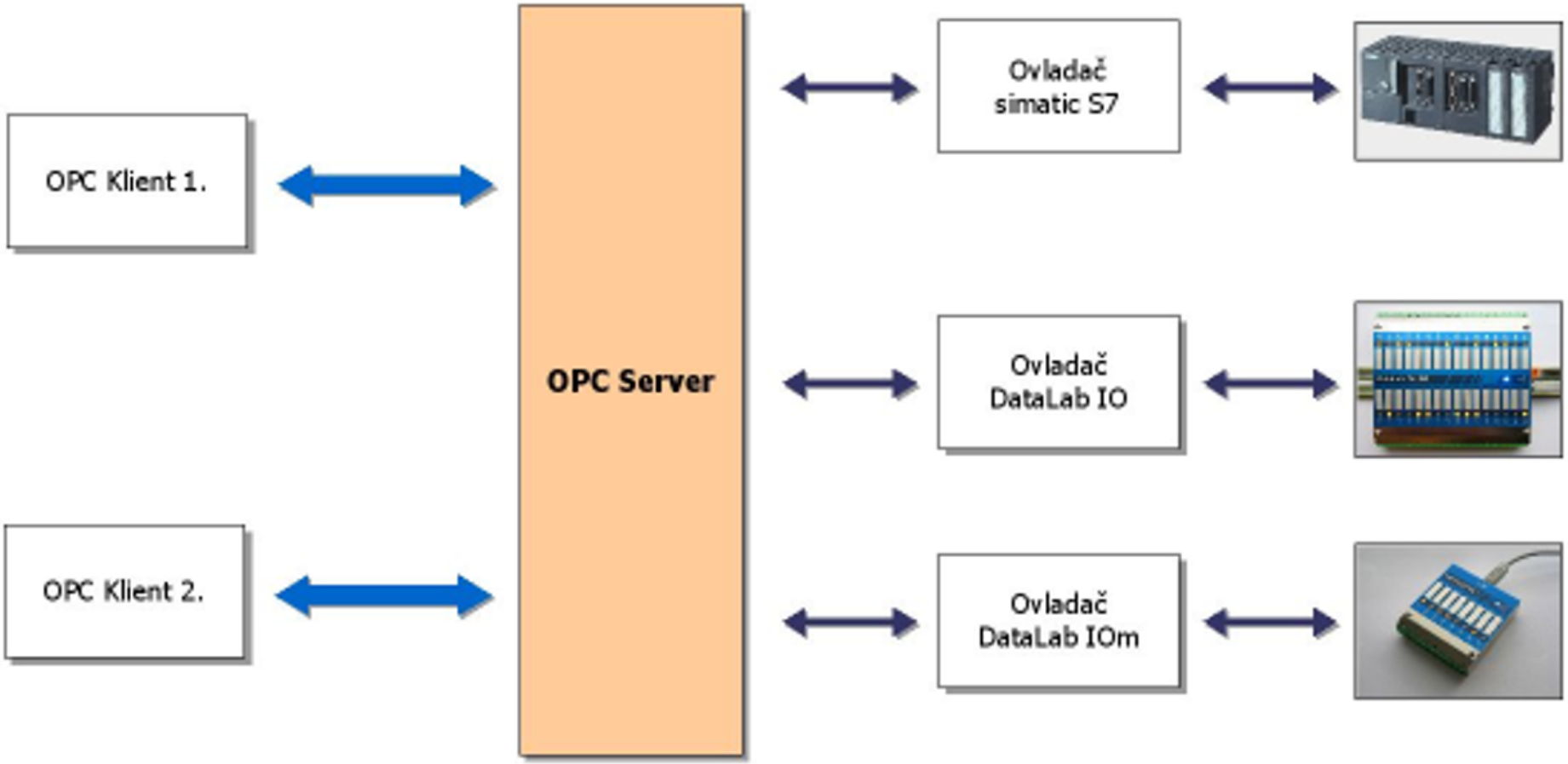
- celý systém nad **více zařízeními**,
- sbírá data z jednotlivých PLC,
- ukládá historii, postmorty, řeší alarmy,
- poskytuje vizualizaci.

SCADA - příklad testovací laboratoř H2



OPC server

OLE for **P**rocess **C**ontrol



OPC server - poskytování dat

Příklad:

- PLC má vlastní způsob poskytování dat: MODBUS, MBUS, PROFIBUS, ...
- SCADA potřebuje data ve své formě, takto zjevně musí obsahovat určeného klienta podle typu komunikace.

Řešení pomocí OPC serveru:

- Externí OPC server se připojí do PLC pomocí určeného typu komunikace, nebo je OPC server přímo součástí PLC.
- OPC server získává data z PLC a ty vystavuje v podobě **uzlů** (= proměnné, uvidíme dále).
- Jakákoliv klientská SCADA se připojuje k OPC serveru, čte tyto uzly a zapisuje je například do HMI.

OPC UA - přístup

- Typicky běží na portu **4840**,
- umožňuje procházení uzlů (**node**), tedy není nutné znát názvy uzlů dopředu, ale vyčtou se ze serveru,
- uzly mají stromovou strukturu (grupování dat),
- uzly mají různé třídy:
  - Root - počátek stromu,
  - Object - runtime data,
  - Type - definice typů,
  - Variable - proměnná, atd.

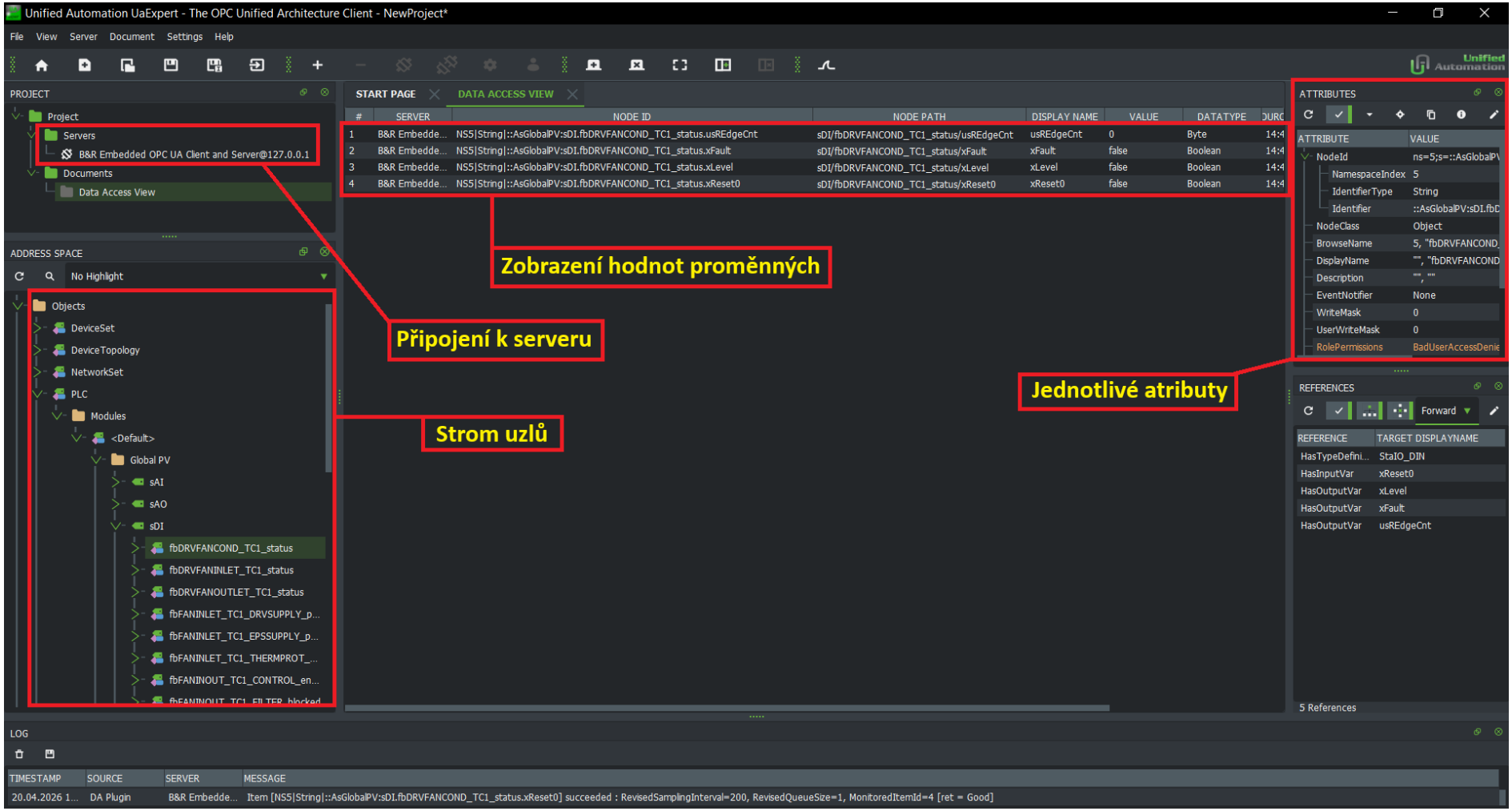
OPC UA - node, uzel

- může jít o proměnnou, metodu, datový typ, ...
- základní atributy:
  - **NodeId** je jednoznačný identifikátor pro komunikaci,
  - **NodeClass** je typ (třída) uzlu,
  - **BrowseName** je název ve stromě při procházení,
  - **DisplayName** je název pro člověka ⇒ lze lokalizovat,
  - **Descripion** je popis,
- další atributy jsou závislé na třídě, například **Variable** bude mít atribut **Value** s hodnotou.

Pro zajištění unikátnosti názvů se zavádí jmenný prostor - **namespace**:

- ns=0, rezervováno pro OPC UA standard, systémové uzly,
- ns=1, obvykle pro server, diagnostika,
- ns=2 a dále pro zařízení.

OPC UA - klient UAExpert



OPC UA - server v Python

```python
import asyncio
from asyncua import Server


async def main():
    server = Server()
    await server.init()

    server.set_endpoint("opc.tcp://127.0.0.1:4840/demo/")
    server.set_server_name("Python OPC UA Demo")

    uri = "urn:demo:python-opcua"
    idx = await server.register_namespace(uri)      # přidělení namespace

    objects = server.nodes.objects
    demo = await objects.add_object(idx, "Demo")    # založení objektu

    teplota = await demo.add_variable(idx, "Teplota", 23.5) # proměnná v objektu demo

    await teplota.set_writable()                    # možnost zapisovat hodnotu z klientské aplikace

    async with server:
        while True:
            v = await teplota.read_value()
            print("Teplota =", v)

            if v >= 50:
                await teplota.write_value(23.5)
            else:
                await teplota.write_value(v + 0.1)

            await asyncio.sleep(1)


if __name__ == "__main__":
    asyncio.run(main())
```

Téma

1 Programování - Build

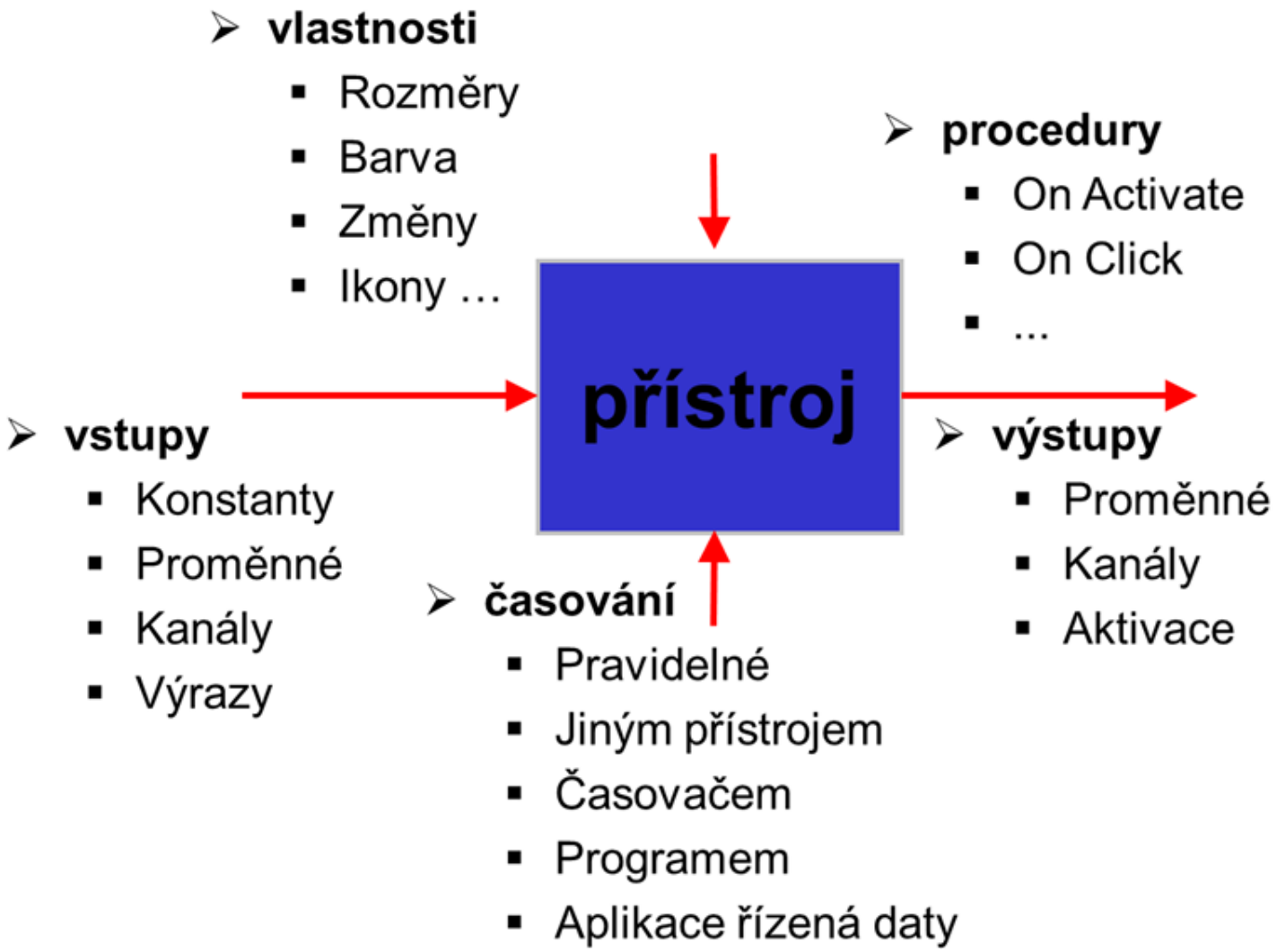
2 HMI, SCADA a OPC

3 Control Web

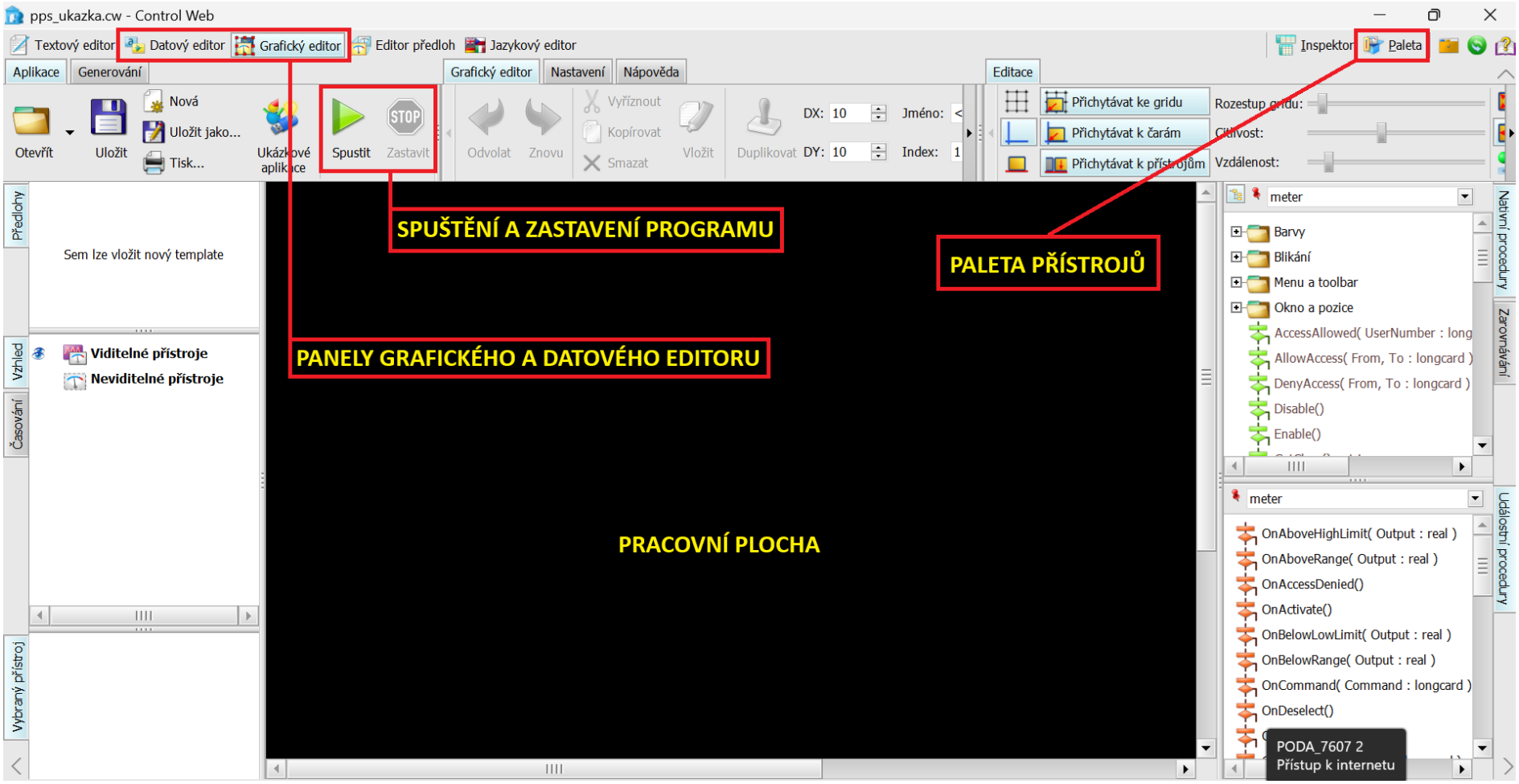
Control Web - systémové vlastnosti

- PC / průmyslová PC
- Windows
- Objektová orientace
- Vizuální programování – jazyk
- Spolupráce s HW – ovladače
- Spolupráce se SW – Active X, DDE, OPC
- Vizualizace v reálném čase, časování objektů

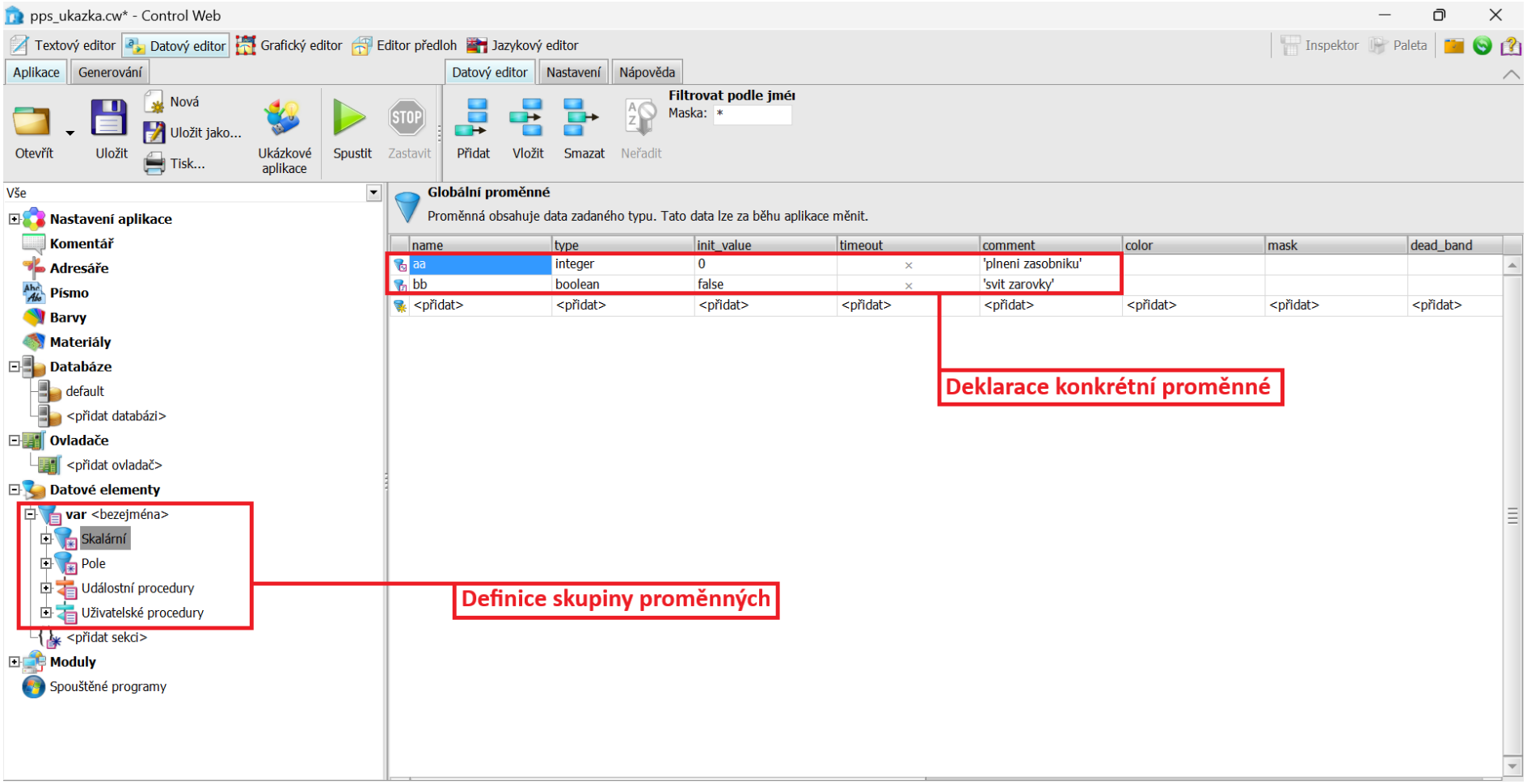
Control Web - přístroj jako prvek programu



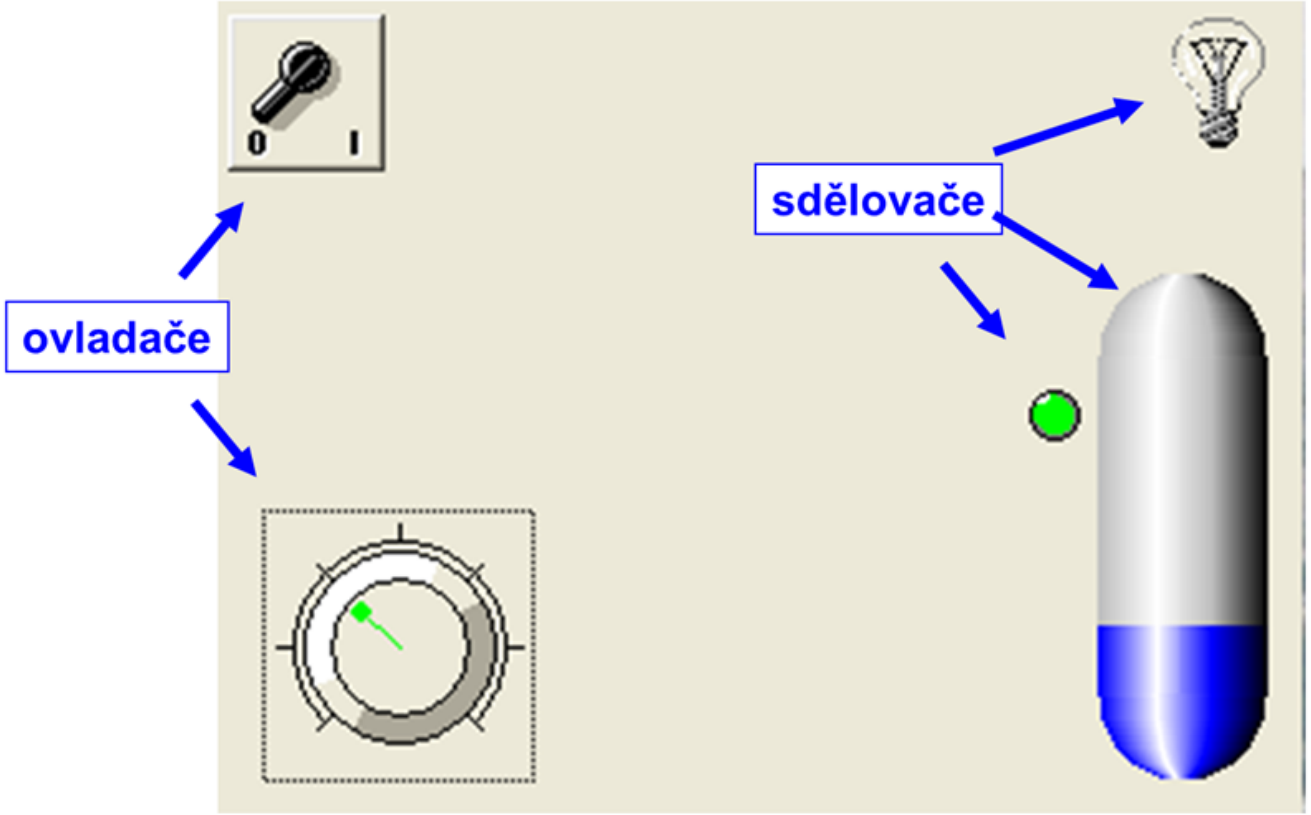
Control Web - hlavní okno programu



Control Web - datový editor

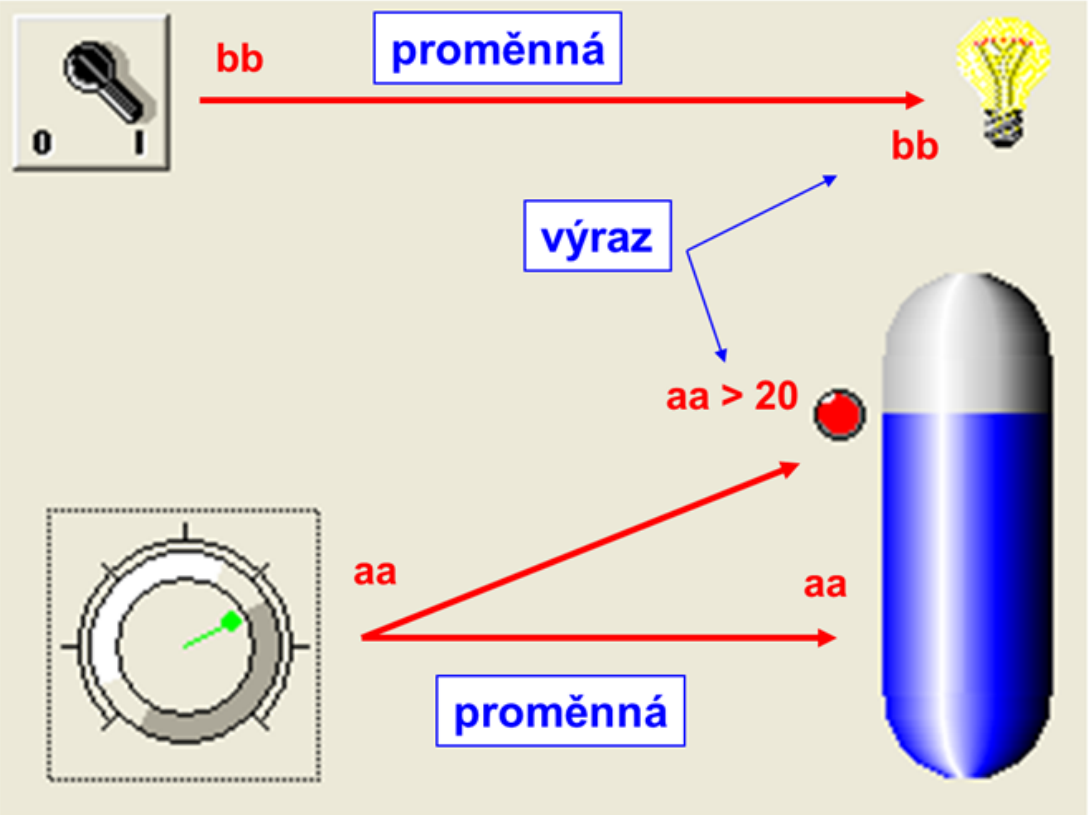


Control Web - objekty



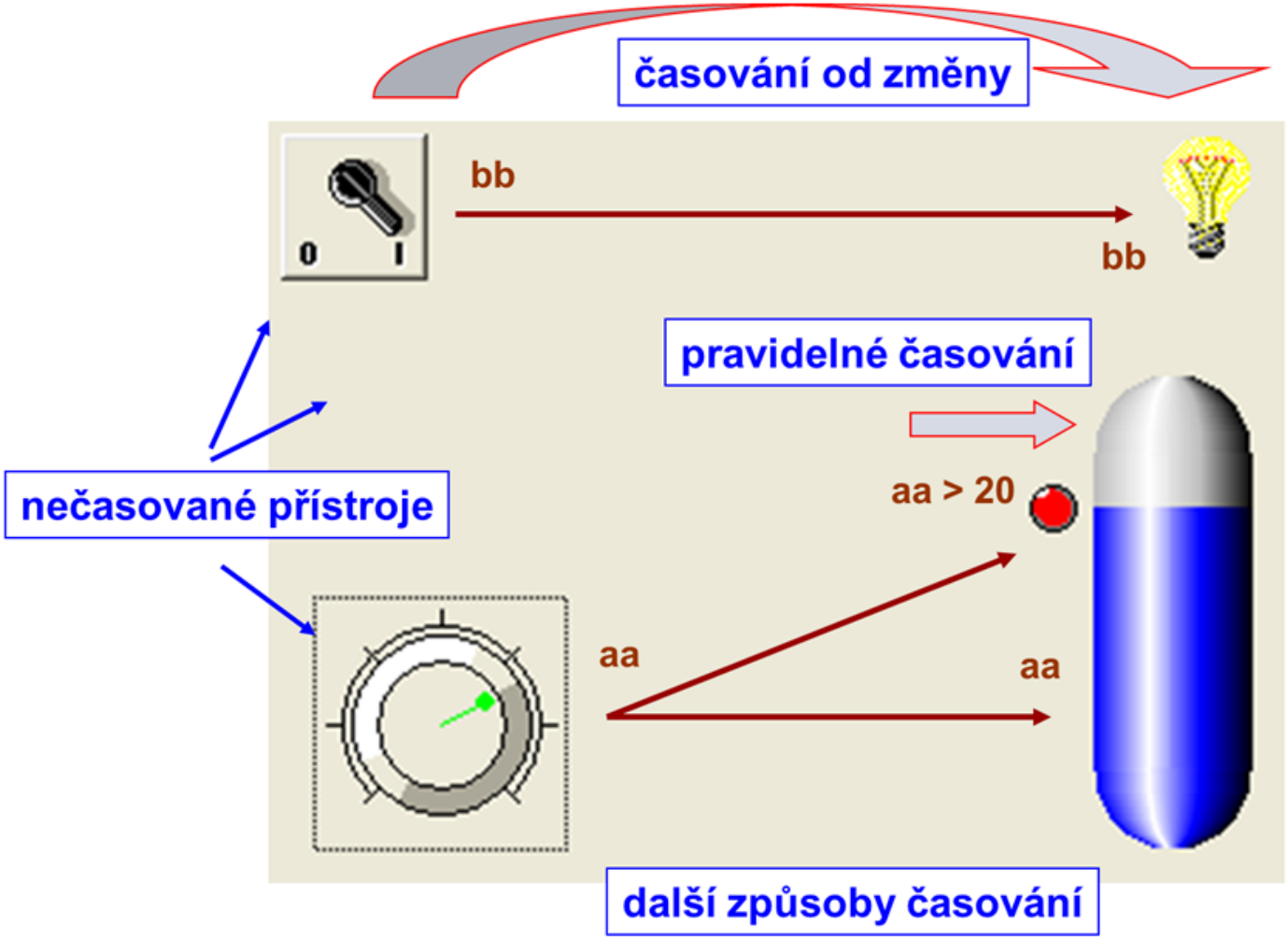
- Přístroje umisťujeme do panelu (šedé pozadí), který reprezentuje hlavní okno aplikace.

Control Web - proměnné



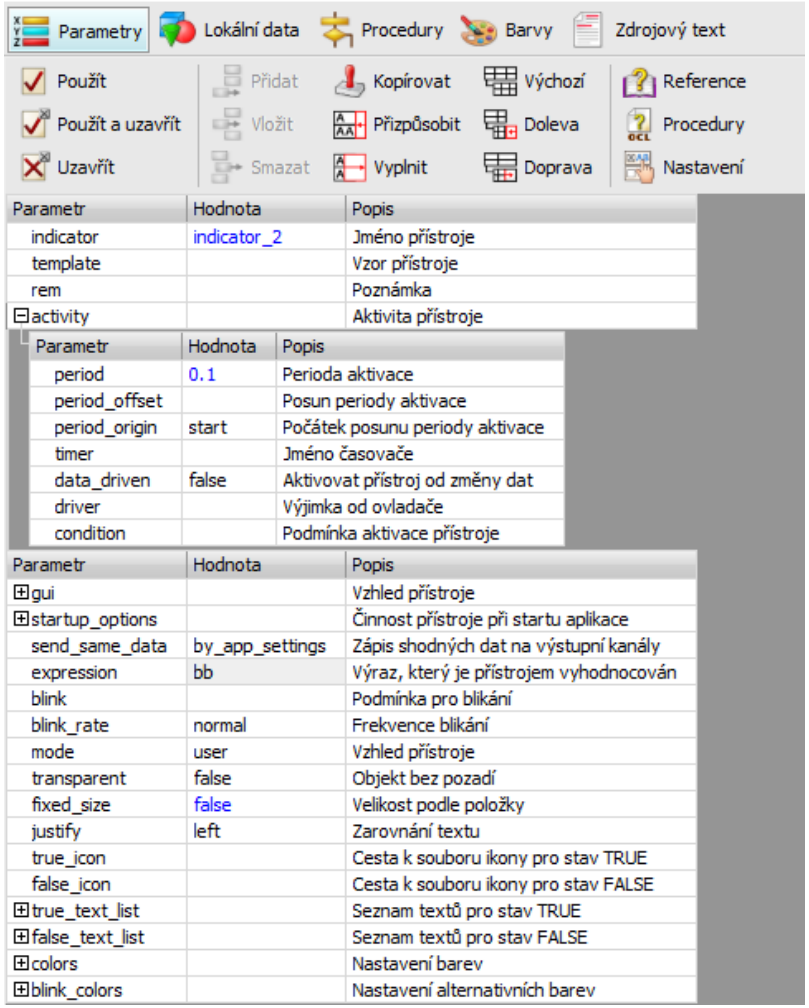
- Přístroje jsou vzájemně propojeny pomocí proměnných.

Control Web - časování



- Časování = stanovuje okamžik vyhodnocení stavu proměnné.

Control Web - časování



- Dvojklikem na přístroji se dostaneme do nastavení parametrů.

Následující přednáška

- Control Web - ovladače,
- Control Web - procedury,
- Control Web - aplikace.

Děkuji za pozornost